PS1 Research / Learning: Computer Vision

This report follows the assignment's original gate and question numbering. The implementation-specific notes describe this repository; general computer-vision concepts are identified separately.

Gate 1 - How does a computer see?

1. What is a digital image? Explain a pixel, image resolution, and the matrix representation of an image.

A digital image samples a scene on a two-dimensional grid and stores a numeric value at each grid location. A pixel (picture element) is one sample. A grayscale pixel stores one intensity, often an 8-bit value from 0 (black) to 255 (white). Image resolution commonly means width by height in pixels; it determines spatial sampling, not by itself the physical size or detail of the original scene.

For an image of height H and width W, grayscale data can be written as I[y,x], with 0 <= y < H and 0 <= x < W. A colour image adds a channel dimension, for example I[y,x,c]. OpenCV's usual colour array order is BGR; many textbooks use RGB. Pixel values are samples and may be affected by exposure, sensor noise, compression, and camera processing.

In this project, `input/official_ps1_road.jpg` is decoded as an H x W x 3 BGR array. The code prints its dimensions, uses grayscale for edge detection, and uses HSV for lane-colour thresholding. Coordinates are scaled from image dimensions so the ROI is not tied to one exact resolution.

2. What are RGB and grayscale images? Why might an autonomous vehicle process them differently?

RGB represents each pixel with red, green, and blue channel intensities. Grayscale represents brightness with one channel. A colour representation can separate cues such as yellow paint from grey asphalt; a grayscale image is cheaper to process and useful for intensity gradients and edges, but discards hue distinctions. Neither is universally superior. Lighting changes can alter both brightness and colour, and a camera's sensor colour channels are not direct measurements of object identity.

For example, a vehicle may threshold yellow lane paint in a colour space, then use grayscale gradients to find boundaries. This PS1 pipeline uses OpenCV BGR input, converts to HSV for a white/yellow mask, and to grayscale for blurred Canny edges. It combines the resulting evidence; it does not infer semantics from colour alone. [1, 2]

Gate 2 - Basic image processing

1. What is Gaussian blur and why is it useful before detecting features?

Gaussian blur smooths an image by convolving it with a Gaussian-weighted kernel. For coordinates x,y and standard deviation sigma, the idealized kernel is G(x,y) = (1/(2 pi sigma^2)) exp(-(x^2+y^2)/(2 sigma^2)). Each output pixel is a weighted average of nearby input pixels. Nearby pixels receive larger weights, so isolated noise and small texture changes are attenuated. The tradeoff is that excessive blur also weakens thin markings and sharp edges.

The implementation applies OpenCV `GaussianBlur` with a 5 x 5 kernel and automatic sigma before Canny. This is a modest fixed smoothing choice for the small supplied image, not a claim that 5 x 5 is optimal for every camera. [3]

2. What is HSV colour space? Why can it sometimes be more useful than RGB for road/lane detection?

HSV encodes hue H (colour family), saturation S (colourfulness), and value V (brightness). OpenCV represents HSV with implementation-specific integer ranges; for 8-bit images its hue is scaled to 0..179 and S,V to 0..255. HSV can make simple colour selection easier than RGB because hue and saturation are represented separately from brightness. For example, yellow paint can be selected by a hue interval, while white paint often has high value and low saturation.

HSV is still affected by shadows, glare, white balance, faded paint, and camera exposure. Hue is unstable when saturation is very low. This code uses a bright, low-saturation range for white and a yellow range with minimum saturation/value, then combines its edges with grayscale edges. The thresholds are explicit in `create_colour_mask()` and may need adjustment. [2]

3. What does Canny edge detection do?

Canny identifies likely intensity boundaries while attempting to suppress noise and keep thin, connected edges. The usual stages are: (1) smooth to reduce noise, (2) compute gradient magnitude and direction, (3) thin responses using non-maximum suppression, (4) classify strong and weak responses using high/low thresholds, and (5) use hysteresis to keep weak responses connected to strong edges. It marks local contrast changes; an edge is not automatically a lane marking.

This pipeline uses thresholds 50 and 150 on the blurred grayscale image, and also computes edges on the threshold mask. The combined edges are restricted to a lower-image ROI before Hough detection. [4]

4. What is a Region of Interest (ROI), and why can it reduce false detections?

An ROI is a selected subset of an image where processing is relevant. Masking out sky, buildings, or distant scenery can reduce irrelevant edge candidates and computational load. The selection encodes an assumption about where the road appears in the camera view; if it excludes a real lane marking, the detector cannot recover it.

Here the ROI is a trapezoid covering the lower road area, with its top at 48% of image height and vertices scaled to width. It is a simple demonstration assumption, not a universal camera calibration. The diagnostic `05_roi_edges.png` displays the retained edges. [5]

Gate 3 - Detecting structure

1. What is a contour? What information can a contour provide?

A contour is an ordered curve, often represented as a sequence of boundary points around a connected foreground region. Depending on the segmentation, contours can describe object outlines and support measurements such as perimeter, area, bounding box, centroid, convex hull, and shape approximation. Contours depend on thresholding and connectivity choices; a broken or merged foreground region changes the result. The current lane pipeline uses Hough segments and polygon filling rather than contour-based lane estimation, though contours could be used to inspect connected paint regions. [6]

2. What is the Hough Transform? Explain how it can be used to find lane lines.

The Hough Transform maps image evidence to a parameter space and accumulates votes for shapes that could explain edge points. A line can be represented in normal form as rho = x cos(theta) + y sin(theta), avoiding infinite slopes. Each edge point votes for possible (rho,theta); peaks in the accumulator indicate lines supported by many points. The probabilistic Hough implementation returns finite line segments rather

than only infinite lines.

For lanes, edge points are first constrained by an ROI; short or implausibly oriented segments can then be rejected. Segments can be grouped by side and combined into a longer boundary estimate. This project calls `cv2.HoughLinesP`, filters by position and slope, then fits x = a*y + b with segment-length weights. Because the official image's center marking curves, the code also samples its bright white pixels by image row and fits a quadratic x(y) for that marking. The opposite edge still uses an explicit width-based inference. [7]

Gate 4 - Lane / path perception

1. Why is lane/path detection useful for an autonomous ground vehicle?

Lane boundaries help estimate the vehicle's lateral position and heading relative to a road corridor. A controller can use that estimate to keep a vehicle within a lane, and a planner can use road geometry to constrain candidate paths. Lane detection is one source of localization and drivable-space evidence; it does not establish that the space is free of hazards.

2. Why can shadows, road texture, poor lighting, missing markings, or clutter create false detections?

Edge and colour methods detect image patterns, not the meaning of those patterns. Shadows create strong brightness transitions; tar seams and aggregate texture create line-like edges; glare can saturate paint and pavement; dusk changes contrast; faded or occluded markings may disappear; and vehicles, vegetation, signs, or road repairs can generate competing lines. Fixed thresholds and a fixed ROI can fail under these shifts. The Hough Transform may accumulate votes for a long road edge or shadow that happens to align with its line model.

This image has one strong white line and a curved rural road. It is therefore a deliberately limited single-photo demonstration. The second boundary in a one-line case can only be inferred under an explicit width/geometry assumption. No accuracy score is claimed without annotated ground truth.

3. Why is lane detection alone not sufficient for a fully autonomous vehicle?

Lane markings may be absent, ambiguous, temporary, or not define a legal/safe path. A vehicle must also localize, detect and track vehicles, pedestrians and obstacles, understand traffic signals and signs, predict motion, plan a safe route, control the vehicle, and handle uncertainty and sensor failures. Redundant sensors, temporal reasoning, safety checks, and broader scene understanding are needed. A classical lane mask from one image is not a complete perception or autonomy system.

Gate 5 - Perspective and BEV

1. What is perspective distortion?

Perspective projection maps 3D points through a camera onto a 2D image plane. Objects farther from the camera appear smaller, parallel road boundaries may converge toward a vanishing point, and equal physical distances occupy different pixel distances at different depths. This affects apparent lane width and slope. A pinhole camera model relates a 3D point to image coordinates through camera intrinsics and depth; calibration is needed to interpret metric geometry. [8]

2. What is a Bird's-Eye View (BEV)? Why can a top-down representation make navigation-related perception easier?

A Bird's-Eye View is a top-down-like representation made by mapping a road-plane image region to a chosen plane. For a planar road and a calibrated or estimated camera, a projective homography H maps image points p to plane coordinates p' up to scale: lambda p' = H p, where lambda is a nonzero scale factor. In OpenCV, `getPerspectiveTransform` computes H from four source and four destination points; `warpPerspective` resamples the image. The source points outline the road patch in the camera view, and destination points define a rectangle in the transformed view.

BEV can make lane widths and lateral offsets more uniform across the image and can simplify geometric navigation reasoning. It cannot correct non-planar road surfaces or poor source-point choices, and it does not restore hidden or absent markings. This PS1 implementation does not perform a BEV warp; it uses image-space ROI and line fitting. A calibrated BEV is an extension, not a result claimed here. [8]

References

1. OpenCV, “Basic Operations on Images,” https://docs.opencv.org/4.x/d3/df2/tutorial_py_basic_ops.html (accessed 2026-10-17).

2. OpenCV, “Changing Colorspaces,” https://docs.opencv.org/4.x/df/d9d/tutorial_py_colorspaces.html (accessed 2026-10-17).

3. OpenCV, “Smoothing Images,” https://docs.opencv.org/4.x/dc/dd3/tutorial_gausian_median_blur_bilateral_filter.html (accessed 2026-10-17).

4. OpenCV, “Canny Edge Detection,” https://docs.opencv.org/4.x/da/d22/tutorial_py_canny.html (accessed 2026-10-17).

5. OpenCV, “Basic Operations on Images” (ROI section), https://docs.opencv.org/4.x/d3/df2/tutorial_py_basic_ops.html (accessed 2026-10-17).

6. OpenCV, “Contours: Getting Started,” https://docs.opencv.org/4.x/d4/d73/tutorial_py_contours_begin.html (accessed 2026-10-17).

7. OpenCV, “Hough Line Transform,” https://docs.opencv.org/4.x/d9/db0/tutorial_hough_lines.html (accessed 2026-10-17).

8. OpenCV, “Geometric Transformations of Images,” https://docs.opencv.org/4.x/da/d6e/tutorial_py_geometric_transformations.html (accessed 2026-10-17).